

Malak

Firmware Development, Programming, Integration, and Calibration

Malak was mainly responsible for the programming and firmware architecture of the Smart Weather Station. She developed and tested the Arduino programs used for the individual sensors and then integrated the required functions into the final SmartWeatherStation.ino firmware.



Her work included implementing the initialization of Serial communication, I²C devices, the LCD, DHT22, BH1750, BMP280, HX711, servo motor, buzzer, button, and water-flow interrupt. She organized the main loop so that the system continuously reads sensors, calculates water flow, filters measurements, updates processed values, controls output devices, refreshes the LCD, and produces CSV data.

Malak implemented the filtering methods used by the project. Temperature and humidity were smoothed to reduce short-term fluctuations, while light used faster smoothing to maintain responsive servo control. Flow measurements were also smoothed, and the load-cell reading used five-reading averaging through `get_units(5)`.

She programmed the servo hysteresis logic. The clothesline opens at or above 105 lux, closes at or below 95 lux, and keeps its previous position between the two thresholds. She also developed the intermittent buzzer warning, which activates when the temperature exceeds 30°C, and the LCD warning message HIGH TEMP!.



Other programmed functions included minimum and maximum temperature tracking, pressure-trend analysis, bright-time accumulation, push-button debounce, navigation between 11 LCD screens, system-status checking, local rule-based forecasting, MQ135 classification, and CSV data output every 60 seconds.

Malak also organized the calibration constants in config.h. The final load-cell factor of -400000 and water-flow factor of 7.6 were separated from the main firmware logic. Calibration was performed jointly with Serena: Malak handled the calibration code, calculations, conversion equations, data processing, and implementation of the final factors, while Serena handled much of the physical test setup and data collection.



Personal reflection:

This project strengthened my knowledge of Arduino programming, modular firmware design, sensor libraries, filtering, interrupts, and non-blocking timing using `millis()`. I learned how to integrate several sensors with different communication methods into one program and how to handle invalid readings without stopping the complete system. I also learned that calibration, filtering, and control thresholds must match the real behavior of the hardware rather than being selected only from theoretical values.

